

Orbital

Live Aircraft, Ship and Satellite Tracking — Project Report

Phone Sett Paing Kyaw (6708369)

Bhone Pyae Hein (6708381)

Han Phyo Htet (6708463)

Nyan Lin Htet (6708397)

Pyae Phyo Maung (6708170)

CSC480 — 15 September 2026

Table of Contents

Chapter 1: Introduction	2
1.1 Background and Problem Statement	2
1.2 Project Objectives	3
1.3 Project Scope	3
1.4 Expected Benefits.....	4
1.5 Tools and Technologies Used	4
1.6 Scale of the System	5
Chapter 2: Feasibility Study and Related Work	6
2.1 Technical Feasibility	6
2.2 Economic Feasibility.....	7
2.3 Operational Feasibility.....	8
2.4 Schedule Feasibility	8
2.5 Related Systems.....	9
2.6 Relevant Theories and Technologies	9
Chapter 3: Requirements Analysis	11
3.1 Stakeholder Analysis	11
3.2 Functional Requirements	12
3.3 Non-Functional Requirements	13
3.4 Use Case Diagram.....	16
3.5 Use Case Descriptions	16

3.6 Software Requirements Specification.....	19
Chapter 4: Project Planning	22
4.1 Project Charter	22
4.2 Work Breakdown Structure.....	23
4.3 Project Schedule	25
4.4 Cost Estimation.....	26
4.5 Resource Planning	27
4.6 Risk Management Plan.....	28
4.7 Communication Plan.....	31
Chapter 5: System Design	32
5.1 System Architecture.....	32
5.2 Database Design	34
5.3 UML Diagrams.....	36
5.4 User Interface Design	39
5.5 Prototype Design.....	40

Chapter 1: Introduction

1.1 Background and Problem Statement

At any moment roughly fourteen thousand aircraft are in the air, tens of thousands of ships are at sea, and thousands of satellites are in orbit. Almost all of them broadcast their position. Aircraft transmit ADS-B, ships transmit AIS, and satellite orbits are published as orbital elements. The data is public and free to collect.

It is difficult to use. Four problems stand in the way.

The raw formats cannot be displayed as received. ADS-B reports altitude in feet and speed in knots. AIS encodes “not available” as a number inside the valid range, so a vessel with no speed reading transmits 102.3 knots. Orbital elements are a text format that must be run through a physics model before it means a position.

Every free source is incomplete, in different places. Measured on identical areas, OpenSky reported 22 aircraft over Myanmar where adsb.lol reported 4; adsb.lol reported 33 over inland China where OpenSky reported none.

Free access is metered. OpenSky bills by area against a daily allowance. adsb.lol permits four requests then roughly one every twelve seconds. Naive polling exhausts either within hours.

Commercial alternatives are closed. MarineTraffic, FleetMon and Spire are now owned by Kpler; ORBCOMM's AIS business by S&P Global.

Problem statement. Public position data for aircraft, ships and satellites is abundant and free, but fragmented across sources with different formats, different coverage gaps and different access limits. No free tool presents all three on one map while being explicit about what it does not know.

1.2 Project Objectives

Objective

- 01 Present live aircraft positions on an interactive Earth in a browser
- 02 Isolate the browser from every external data source
- 03 Survive upstream failure without breaking the display
- 04 Normalise every source onto one shared data shape
- 05 Extend the same architecture to satellites and ships
- 06 Provide user accounts and a differentiated service tier
- 07 Record the reasoning behind every significant decision

05 exists to test whether the abstraction in 04 is real or accidental.

1.3 Project Scope

In scope

Area	Included
Aircraft	Live positions from two sources, merged; flight track; airline and type; airport search; receiver coverage
Satellites	~1,400 satellites computed from orbital elements; orbit class; the orbit each one is on
Ships	~26,000 vessels from two sources, merged; type, dimensions, destination, status
Solar system	Eight planets and the Moon, navigable
Accounts	Registration, sign-in, sessions, three tiers
Time travel	Historical satellite positions, limited by tier

Out of scope

Excluded	Reason
Space debris and rocket bodies	The catalogue holds ~100,000 objects, most not worth drawing
Collision or re-entry prediction	The model is accurate to kilometres; a viewer could not detect a wrong answer
A fourth object class	Requires a written decision; enforced by a test on each side

Ground-station passes	Feasible but outside the objectives
Flight or voyage prediction	Orbital reports observations, it does not forecast

1.4 Expected Benefits

For viewers. One free view of aircraft, ships and satellites, with no account required and no claim of completeness the data cannot support. Markers older than two minutes fade. The ship layer is labelled “ships in coastal waters” because both AIS feeds are received from the shore.

For data providers. One backend makes a fixed number of requests regardless of how many people are watching, polling each source at the rate that source asks for.

For the team. A worked example of a system with three interchangeable data layers, where the interchangeability was tested twice by adding a layer. Each new layer cost one provider module and one registry entry.

1.5 Tools and Technologies Used

Layer	Technology	Reason
Map	MapLibre GL JS	One continuous view from globe to street level on real imagery
3D models	three.js	Draws the selected aircraft inside MapLibre’s WebGL context
Frontend	React, TypeScript, Vite	Contract enforced at compile time
Backend	FastAPI, Python	Pydantic makes the same contract executable
HTTP	httpx	Async, so polling never blocks a request
WebSocket	websockets	The global AIS feed pushes rather than answers
Storage	In-process memory; SQLite for accounts	One process, one poller; accounts are the only state that must survive a restart
Orbits	sgp4	The standard propagator for the published element format
Tests	pytest, Vitest	Both run offline against fixture data

Data sources

Source	Provides	Access
adsb.lol	Aircraft	Open Database Licence, no key
OpenSky Network	Aircraft	Free, non-commercial, OAuth2, metered
Digitraffic (Fintraffic)	Ships, Baltic	CC BY 4.0, commercial permitted, no key
aisstream.io	Ships, global	Free, key required

CelesTrak / SatNOGS	Orbital elements	Free
NASA GIBS, OpenFreeMap	Imagery and map tiles	Open

1.6 Scale of the System

Measure	Value
Backend	~19,400 lines, 85 files
Frontend	~31,900 lines, 167 files
Automated tests	1,868 (832 backend, 1,036 frontend)
HTTP endpoints	17
Recorded decisions	169
Objects served	14,804 aircraft, 26,262 ships, 1,427 satellites

Measured on 8 September 2026 against the commit this report was built from. The three object counts move continuously; the rest move whenever the code does, which is why this table carries a date.

Chapter 2: Feasibility Study and Related Work

2.1 Technical Feasibility

Every claim below was tested against the live services before the design depended on it.

Question	Result
Are live aircraft positions obtainable without payment?	Yes. OpenSky returned 11,651 worldwide; adsb.lol 10,009
Can the whole world be fetched economically?	Yes. 10,013 aircraft in 1.9 seconds
Can satellite positions be computed rather than fetched?	Yes. ~1,400 satellites, no network in the request path
Are live ship positions obtainable without payment?	Yes. 916 vessels in 37 KB from one request; 17,848 in four minutes from the stream

Two measurements that changed the design

One global query is not enough. A single 6,000 nautical-mile circle returned 10,013 aircraft and four circles returned 10,009, which appeared to prove one was sufficient. The other three sample points were inside the area the first already covered. Over Australia the single circle returned 0 and a direct query returned 27. The global sweep uses four circles.

A rate limit cannot be characterised with a burst. adsb.lol appeared to tolerate any rate when tested with occasional bursts. Under continuous load it refused 38% of requests. The real limit is four requests, then approximately one every twelve seconds.

Performance

The backend runs a single-threaded event loop, so a slow operation delays every other request.

Objects held	Poll apply	Bbox filter	Thin to 2,000	Search
2,000	0.7 ms	0.57 ms	0.82 ms	1.6 ms
10,000	7.4 ms	3.2 ms	9.1 ms	8.3 ms
30,000	14.4 ms	9.2 ms	43.5 ms	24.2 ms

Rendering at the 2,000-marker cap costs 0.80 ms per update, 4.8% of a 60 fps frame budget, scaling linearly to 10,000.

Conclusion: technically feasible. Every required capability was demonstrated on live services before being designed around.

2.2 Economic Feasibility

Direct cost

Category	Cost
Data sources	0
Map imagery and tiles	0
Libraries and tooling	0
Hosting during development	0
Total	0

The real cost is labour: five students over one semester.

The quota is the budget

No money changes hands, but OpenSky's daily credit allowance is finite and was budgeted like money. The polling intervals are derived from it rather than chosen by feel: the configured jobs project 3,072 credits per day against a 4,000 allowance, 77%, leaving headroom for the lookups a user triggers by clicking. The system refuses to start if a configuration would exceed 85%.

Using the free adsb.lol feed as the primary source, with OpenSky supplementing it every 120 seconds, decouples the refresh rate from the allowance entirely. Ships and satellites cost nothing.

Licensing constrains the revenue model

Source	Commercial use
Digitraffic	Permitted (CC BY 4.0)
adsb.lol	Permitted (ODbL)
OpenSky Network	Forbidden, non-commercial only
aisstream.io	Not stated

A commercial version is possible but not with the current source mix: the aircraft layer would have to drop OpenSky and accept its coverage gaps.

The implemented model

A free tier supported by advertising and a paid tier are both implemented. Advertisements are the project's own messages in real advertisement dimensions, because a third-party network would introduce tracking into a project that collects no analytics. There is no payment processor, so the button reads "Switch premium on" rather than "Buy", and a test enforces that wording.

2.3 Operational Feasibility

Operation

Orbital runs as two processes with no database server, no message queue and no external cache. State lives in process memory; only accounts must survive a restart, in one SQLite file.

Failure behaviour

The API layer never calls an upstream service. It reads a cache the ingestion layer fills on a schedule, so there is no upstream call in the request path that can fail.

Failure	Behaviour
Upstream 5xx	Last good snapshot served, flagged stale; poller backs off
Rate limited	Same, using the upstream's own retry hint
One of two feeds down	The other continues; the map is drawn
CelesTrak returned 503 for a full day	The SatNOGS fallback served; cached elements remain usable for days
Network absent	The fixture provider replays committed data

Readability

The interface must not make claims the data cannot support. Markers older than two minutes fade; an object with no transmitted heading is drawn as a disc rather than an arrow pointing north; the ship layer names its actual coverage; and the satellite layer reports no data age, because a computed position does not have one.

2.4 Schedule Feasibility

Work proceeded in phases, each with an explicit boundary, and each session ended with a written handover.

Phase	Delivered
1	Aircraft layer end to end
2	Satellite layer; migration to MapLibre
3	Moon and solar system views
4	Accounts, tiers, advertisements
5	Ship layer, regional then global
6	Performance and defect sweep

The strongest evidence that the schedule held is the cost of the later phases. The satellite layer cost one provider module and one registry entry; the ship layer cost the same four months later; and the polling logic was not modified for either.

Where time was lost: six sessions went to one defect misdiagnosed five times. The cause was structural — one camera serving two pictures at very different scales — and the resolution deleted 376 lines, added 130, and cost no feature.

2.5 Related Systems

System	Covers	Model	Relationship
Flightradar24	Aircraft	Commercial	The reference for a mature tracker; larger network, closed data
MarineTraffic	Ships	Commercial	The shipping equivalent; acquired during this project
OpenSky Network	Aircraft	Research	Used as a source; its non-commercial clause constrains §2.2
adsb.lol	Aircraft	Community	Used as the primary aircraft source
AISHub	Ships	Reciprocal	Rejected: access requires contributing a receiver
N2YO, Heavens-Above	Satellites	Free	Comparable satellite tracking; neither combines classes

Each individual layer is done better elsewhere by organisations with larger networks. What distinguishes Orbital is that three classes of object with three different acquisition models — observed and metered, observed and free, and computed — are presented on one map through one shared data shape.

A deliberate search found no free, global, key-less ship feed equivalent to adsb.lol. An AIS receiver is a real installation rather than a cheap dongle, so volunteer networks operate on reciprocity, and the commercial half of the market has consolidated under two owners.

2.6 Relevant Theories and Technologies

ADS-B. Aircraft broadcast identity, position, altitude and velocity about once a second. Coverage depends on a receiver being in range, which is why oceanic regions are sparse in every terrestrial network.

AIS. Vessels broadcast position every few seconds and identity — name, type, dimensions, destination — every six minutes. Two consequences follow. “Not available” is a number rather than a blank: a speed of 102.3 knots, a course of 360° and a heading of 511 all mean the transmitter did not say. And identity accumulates far more slowly than position: measured at 26.5 static messages per second against 114 position reports, vessel type coverage rose from 3% at thirty seconds to 62% after fifty-four minutes.

SGP4 and orbital elements. Satellite positions are computed from published two-line element sets. Elements describe an orbit rather than a position and stay usable for days, so the layer survives an upstream outage. Accuracy degrades roughly a kilometre per day from epoch, silently: of 1,670 element sets in one live feed, 87 were over a year old and the oldest was from 1975. Anything past seven days is refused.

Map projection. A pixel covers a different ground distance at every zoom and latitude — about 750 m at zoom 7, about 6 m at zoom 14. This arithmetic determines how often the display needs redrawing, and it is why drawing every object at every zoom is neither useful nor affordable.

Signed distance field icons. Marker symbols are stored as distance fields so one silhouette can be tinted per object rather than needing an image per colour. The alpha channel encodes distance rather than coverage, so thin detail dissolves when scaled down; marker artwork is therefore deliberately chunky.

HTTP conditional requests. The polled endpoints use ETags. When the data has not changed the server returns 304 Not Modified before reading its store — no filtering, thinning, serialisation or compression.

Chapter 3: Requirements Analysis

3.1 Stakeholder Analysis

Orbital's stakeholder map has an unusual shape: most of its stakeholders are upstream. The parties with the greatest power over whether the system works are the data providers, none of whom have any relationship with the project.

Upstream — the data providers

Stakeholder	Interest	Power	How the design responds
adsb.lol	That clients do not abuse a donated service	High	One backend, fixed request count regardless of viewers; a gate enforces the measured rate limit
OpenSky Network	Non-commercial use; quota respected	High	Used as a supplement; spending held at 77% of allowance; startup refuses an overspending configuration
Fintraffic	Attribution under CC BY 4.0	Medium	Attributed; polled at the rate its own cache headers ask for
aisstream.io	Not stated	Medium	Used as a free public service and credited; behind a switch that disables it if the project is monetised
CelesTrak / SatNOGS	Not to be polled excessively	Medium	Refreshed every six hours, cached to disk, with a fallback
Volunteer receiver operators	Recognition that coverage is theirs	Low	The map draws where no network reaches

Because the upstream stakeholders hold the power and grant no guarantees, the system is designed so any of them failing degrades the display rather than breaking it. That is a requirement derived from stakeholder analysis rather than a technical preference.

Downstream — the users

Stakeholder	Needs	Power
Anonymous viewer	To look without registering; not to be misled	High, the default user
Registered free user	An identity; recent history	Medium
Premium user	A wider historical window	Medium
Administrator	Account management	Low, one role

The administrator tier exists in the data but has no interface. An account can only be promoted from a terminal on the server, and the public endpoint refuses that tier under all configurations.

The team

Member	ID	Roles
--------	----	-------

Phone Sett Paing Kyaw	6708369	Project Manager, Developer
Bhone Pyae Hein	6708381	System Analysis, Co-Developer
Han Phyo Htet	6708463	Quality Assurance, Business Analysis, Co-Tester
Nyan Lin Htet	6708397	Technical Engineer, Tester
Pyae Phyo Maung	6708170	Co-Tester, Co-Quality Assurance and Analysis

3.2 Functional Requirements

Priority is M (must), S (should) or C (could). All are implemented.

Data acquisition

ID	Requirement	Pri
FR-01	Retrieve live aircraft positions from an external source	M
FR-02	Merge two aircraft sources, preferring the fresher record	S
FR-03	Compute satellite positions from orbital elements	M
FR-04	Refuse orbital elements older than seven days	M
FR-05	Retrieve ship positions from a regional and a global source	S
FR-06	Merge ship sources so a field is never overwritten by a blank	M
FR-07	Poll each source on a schedule within its rate limit	M
FR-08	Retain the last successful snapshot when a source fails	M
FR-09	Evict objects not re-observed within a per-layer TTL	M
FR-10	Run entirely offline from fixture data	M

API

ID	Requirement	Pri
FR-11	List objects of a layer, filtered by bounding box	M
FR-12	Return one object in full, with its observed track	M
FR-13	Search a layer by name or identifier	M
FR-14	Search all four kinds in one request	S
FR-15	Reduce results to a maximum, spread across the visible area	M
FR-16	Answer 304 when the client holds the current representation	S
FR-17	Report ingestion health and data freshness	M
FR-18	Never call an external service while handling a request	M
FR-19	Refuse malformed input with a precise error	M
FR-20	Clamp a requested result count to the configured maximum	M

Presentation

ID	Requirement	Pri
FR-21	Draw an interactive Earth, globe to street level	M

FR-22	Draw each object oriented to its heading	M
FR-23	Draw an unknown heading without implying one	M
FR-24	Fade an object older than two minutes	M
FR-25	Interpolate positions between polls	M
FR-26	Switch between the three layers	M
FR-27	Change all surrounding text and the key with the layer	M
FR-28	Select an object and show its details	M
FR-29	Draw the observed track of the selected object	M
FR-30	Draw the selected aircraft as a 3D airframe	C
FR-31	Show receiver coverage	S
FR-32	Offer a solar system view with navigable planets	C
FR-33	Colour each layer by the property that varies within it	M

Accounts and entitlements

ID	Requirement	Pri
FR-34	Register with an email and a password of at least 10 characters	M
FR-35	Sign in and out; maintain a session across reloads	M
FR-36	Hash passwords; never store them in plain text	M
FR-37	View satellite positions at a past instant	S
FR-38	Limit the window by tier: 24 hours free, 7 days premium	M
FR-39	Refuse a request outside the window with a stated reason	M
FR-40	Allow an account to change its own tier between free and premium	S
FR-41	Refuse the administrator tier from the public interface	M
FR-42	Show advertisements to accounts that are not premium	S

The premium window is seven days because that is where propagation accuracy becomes untrustworthy. No tier can exceed it.

3.3 Non-Functional Requirements

Performance

ID	Requirement	Target	Measured
NFR-01	Marker rendering at the cap	< 16.7 ms	0.80 ms at 2,000 markers
NFR-02	Sustained frame time under load	60 fps	4.2 ms p50, 5.9 ms p99
NFR-03	List endpoint response	< 100 ms	23 ms
NFR-04	Combined search response	< 100 ms	47 ms

NFR-05	Detail lookup	< 10 ms	0.01 ms
NFR-06	Responses compressed	—	366 KB to 24.5 KB
NFR-07	No request may block the event loop indefinitely	—	Result count clamped

Reliability

ID	Requirement
NFR-08	An upstream failure must not produce an error response
NFR-09	Data past its threshold must be flagged, not hidden
NFR-10	One source failing must not disable a multi-source layer
NFR-11	A dropped stream must reconnect without losing accumulated state
NFR-12	The system must start without credentials

Accuracy

ID	Requirement
NFR-13	A protocol's "not available" encoding must never be shown as a value
NFR-14	An object without a position is not drawn
NFR-15	An unknown heading must not be drawn as north
NFR-16	Values shown in the unit their readers use
NFR-17	Text around the map must describe the layer shown, including its limits
NFR-18	A computed position must not report an observation age
NFR-19	A physically impossible speed must be rejected

Security

ID	Requirement
NFR-20	Passwords hashed with scrypt; never logged or returned
NFR-21	Session cookies HttpOnly and SameSite
NFR-22	Credentials read from environment or a git-ignored file
NFR-23	A configuration object must never be printed whole
NFR-24	Administrator privilege not obtainable through any public endpoint

Maintainability

ID	Requirement	Evidence
NFR-25	One shared data shape across all layers and both languages	Kept in step by type checks
NFR-26	A new data source must not require frontend changes	Demonstrated twice
NFR-	A new object class requires a written decision	Enforced by a test on each

27

NFR-28 Every significant decision recorded

side

169 entries

NFR-29 Both suites runnable offline

No test makes an external call

Usability

ID	Requirement
----	-------------

NFR-30	Usable without an account
--------	---------------------------

NFR-31	Reduced-motion preference respected
--------	-------------------------------------

NFR-32	Controls reachable and labelled for assistive technology
--------	--

NFR-33	Layout usable at a 375 px viewport
--------	------------------------------------

3.4 Use Case Diagram

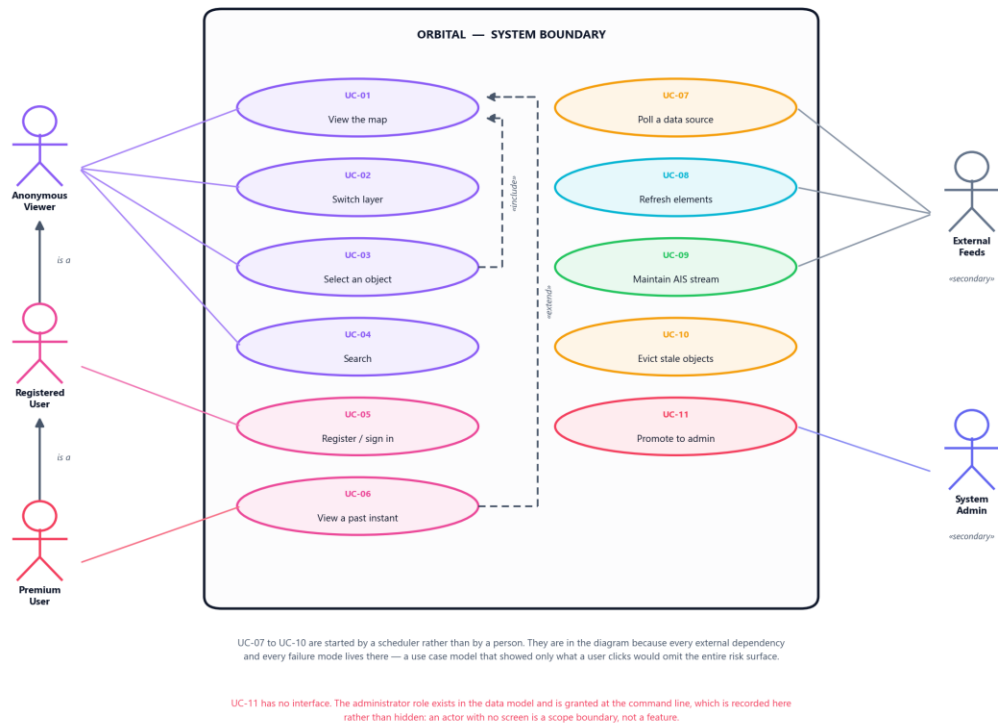


Figure 3.1 — Use case diagram. Eleven use cases, five actors, one system boundary.

UC-03 includes UC-01. UC-06 extends UC-01 when an instant is chosen, if that instant lies within the actor's entitlement window. UC-07 to UC-10 are started by a scheduler rather than a person, and are shown because every external dependency and failure mode lives there.

3.5 Use Case Descriptions

UC-01 View the map

Actor Anonymous Viewer. **Preconditions** None; no account required.

1. The system draws the Earth on satellite imagery.
2. The client requests the active layer's objects for the visible area.
3. The API returns the cached snapshot, filtered and thinned, with its age.
4. The client draws each object oriented to its heading, coloured by the property that varies on that layer.
5. Between polls the client interpolates positions.

A1 Data is stale: the response is flagged, markers past two minutes fade. **A2** Backend unreachable: the last state remains; the map is not cleared. **A3** More objects than the cap: a spatially even sample is returned and both figures are reported.

UC-02 Switch layer

Actor Anonymous Viewer.

1. The viewer selects Aircraft, Satellites or Ships.
2. The system clears the current selection and objects.
3. All surrounding text changes with the layer — subtitle, search placeholder, count noun and colour key.
4. Layer-specific furniture is hidden; airports and receiver coverage belong to the aircraft layer.
5. The client polls the new layer's endpoint.

UC-03 Select an object

Actor Anonymous Viewer. **Includes** UC-01.

1. The viewer clicks a marker.
2. The client requests that object's full record.
3. A panel opens with fields appropriate to the layer.
4. The object's observed track is drawn.

A1 Empty map clicked: the selection clears. **A2** No longer tracked: a 404 is returned and the panel says so rather than showing an empty record.

UC-04 Search

Actor Anonymous Viewer.

1. The viewer types into the search box.
2. After a debounce, the client requests results.
3. Matches are returned in separate groups, ranked within each but not across them, because they are not comparable.
4. Only the groups the active layer can act on are shown.
5. Choosing a result moves the camera to it and selects it.

UC-05 Register and sign in

Actor Anonymous Viewer becoming Registered User.

1. The viewer supplies an email and password.
2. A password shorter than ten characters is rejected.
3. The password is hashed with bcrypt.
4. A session cookie is issued, HttpOnly and SameSite.

A1 Email already registered: reported without revealing the password. **A2** Wrong credentials: one message for both cases, so the response does not disclose which accounts exist.

UC-06 View a past instant

Actor Registered or Premium User. **Extends** UC-01. **Precondition** Signed in, satellite layer active.

1. The user moves the time control to a past instant.
2. The system checks it against the account's window: 24 hours free, 7 days premium.
3. Positions are propagated to that instant and returned.

A1 Outside the window: refused with the window that does apply. **A2** Beyond seven days: refused for every tier, because propagation accuracy degrades about a kilometre per day.

UC-07 Poll a data source

Actor Scheduler, with an external feed as secondary actor.

1. The scheduler waits the configured interval.
2. It checks the remaining quota and the rate gate; if either forbids, it skips.
3. The provider fetches and converts to the shared shape, discarding objects without a usable position and converting every "not available" encoding to an explicit absence.
4. The store merges the result.

A1 Unreachable: the previous snapshot is retained and the next attempt is delayed by an increasing backoff. **A2** Rate limited: the upstream's own retry hint is used. **A3** Both sources of a layer fail: only then is it an outage.

UC-08 Refresh orbital elements

1. Every six hours elements are fetched from CelesTrak.
2. Sets older than seven days from epoch are refused.
3. Accepted elements are cached to disk.

A1 CelesTrak unavailable: SatNOGS is used. If both fail, cached elements continue to serve.

UC-09 Maintain the global AIS stream

1. A WebSocket is opened and subscribed to the whole world.
2. Positions and identities are accumulated in memory.
3. Positions older than fifteen minutes are removed; identities are kept for six hours, because a vessel's name does not expire when its position does.

A1 Connection dropped: reconnection with backoff, losing nothing, because the accumulated world is held locally.

UC-11 Promote an account to administrator

Actor System Administrator at a terminal on the server.

1. The administrator runs the account tool on the host.
2. The tool sets the account's tier.

There is no interface for this and no endpoint that can perform it.

3.6 Software Requirements Specification

Purpose

This specification covers Orbital, a web application presenting live aircraft, ship and satellite positions on an interactive Earth. It defines the external interfaces, the shared data contract and the operating constraints.

System context

External feeds (six sources) -> 1. INGESTION providers, poller, store -> 2. API reads the store only -> 3. FRONTEND MapLibre and React client

Data flows in one direction, and each layer knows only the layer beneath it. Three properties follow:

1. The browser never talks to a data source, so rate limiting and caching are enforced in one place.
2. Ingestion does not know a browser exists, and the frontend does not know where the data came from.
3. Upstream failure is contained at layer 1.

User classes

Class	Access
Anonymous Viewer	Full read access, no account
Registered User	Identity, 24-hour history
Premium User	7-day history, no advertisements
Administrator	Terminal only

Constraints

Constraint	Source
OpenSky may not be used commercially	Provider terms
OpenSky spending within a daily allowance	Provider terms
adsb.lol allows 4 requests then ~1 per 12 s	Measured
Digitraffic states a 60-second cache	Provider headers
Satellite accuracy untrustworthy beyond 7 days	SGP4
The backend is a single-threaded event loop	Runtime
AIS identity arrives on a 6-minute cycle	Protocol

API

Method	Path	Purpose
GET	/api/aircraft	Aircraft in a bounding box
GET	/api/aircraft/{id}	One aircraft with track and route

GET	/api/aircraft/search	Aircraft by callsign or address
GET	/api/satellites	Satellites, optionally at a past instant
GET	/api/satellites/{id}	One satellite
GET	/api/ships	Ships in a bounding box
GET	/api/ships/{id}	One ship
GET	/api/ships/search	Ships by name or MMSI
GET	/api/search	Combined search across four kinds
GET	/api/moon/satellites	Spacecraft in lunar orbit
GET	/api/health	Ingestion status and freshness
POST	/api/auth/register	Create an account
POST	/api/auth/login	Begin a session
POST	/api/auth/logout	End a session
GET	/api/auth/me	The current account
POST	/api/auth/subscription	Change own tier

The shared data shape

Every moving object, whatever its source, is represented identically.

Field	Type	Meaning
id	string	Stable identifier within a provider
lat, lon	number	Degrees, WGS84
altitude	number or null	Metres above mean sea level; zero means zero, null means the source did not say
velocity	number or null	Metres per second
heading	number or null	Degrees clockwise from true north; null is never drawn as north
label	string	Short human-readable name
model	string or null	What the source says the object is
lastSeen	string	When the position was current, never when we polled
type	enum	aircraft, satellite or ship

Anything meaningful to only one kind of object lives in a generic meta map, so a provider can add a field without a frontend change. This is what made the second and third layers cost one module each.

Verification

Class	Method
Functional	1,868 automated tests; live runs against every real source

Performance	Instrumented measurement in the browser and a backend benchmark
Reliability	Injected outages, plus real ones that occurred unprompted
Accuracy	Unit tests on each conversion, plus inspection of the running system
Security	Tests on hashing, session handling and tier refusal

Eighteen defects are recorded as having been invisible to the tests at the time they existed, each found by running the application and looking at it. In every case the code was correct and the wiring was absent or mismatched. Unit tests verify code; only running the system verifies wiring.

Chapter 4: Project Planning

4.1 Project Charter

Purpose

Orbital exists to put aircraft, ships and satellites on one map, in a browser, for free, with no account required. The three kinds of object are already tracked publicly and separately; nothing assembles them into a single view a person can simply open.

Objectives

#	Objective	How it will be judged
01	One map showing all three object types	All three layers render live data from the deployed system
02	No account and no cost to look	The default view requires no sign-in and no payment
03	Upstream failure degrades, never breaks	With a feed disabled, the map still draws, labelled stale
04	A new object type costs one module	Adding a type requires no change to the API or the frontend
05	Honest reporting of what is shown	Sampling, staleness and source are stated in the interface

Scope

In scope. Ingestion from free public feeds; normalisation to one shared shape; a read-only REST API; a browser client rendering the three layers on a map; object selection and detail; search; accounts with a free and a premium entitlement window; the Moon, the eight planets and the solar system view.

Out of scope. Writing to any upstream; historical archiving beyond the retention window; native mobile applications; flight booking, ticketing or any transaction; real-time alerting or push notification; an administrator interface.

Deliverables

The deployed application, the source repository with its test suites, this report, a decision record, a test plan, and a demonstration.

Constraints

Constraint	Consequence for the plan
Free data sources only	Rate limits, not money, are the budget (§4.4)
No commercial licence on one feed	The revenue model cannot use OpenSky (§2.2)
One backend worker	Vertical scaling only; a second worker would double

	upstream cost
Five part-time students	Parallel work must be separable, or it serialises
One semester	Scope is fixed by phase boundaries rather than by deadline pressure

Assumptions

That the public feeds remain available and free; that their terms do not change mid-project; that free hosting tiers remain sufficient for a demonstration workload. **The first of these did not hold** — OpenSky became unreachable from every cloud host partway through — and §4.6 treats it as the realised risk it turned out to be rather than as an assumption that was merely unlucky.

Authority

The Project Manager owns scope and phase boundaries and decides what is in the current phase and what is explicitly out. Every significant decision is written to the decision record with its alternatives and its reasoning, so that a choice can be defended, or reversed, on the evidence that produced it.

4.2 Work Breakdown Structure

The work is broken down to the level at which one person owns one deliverable. Decomposing further produces a task list that is stale within a week; decomposing less leaves nobody accountable for anything.

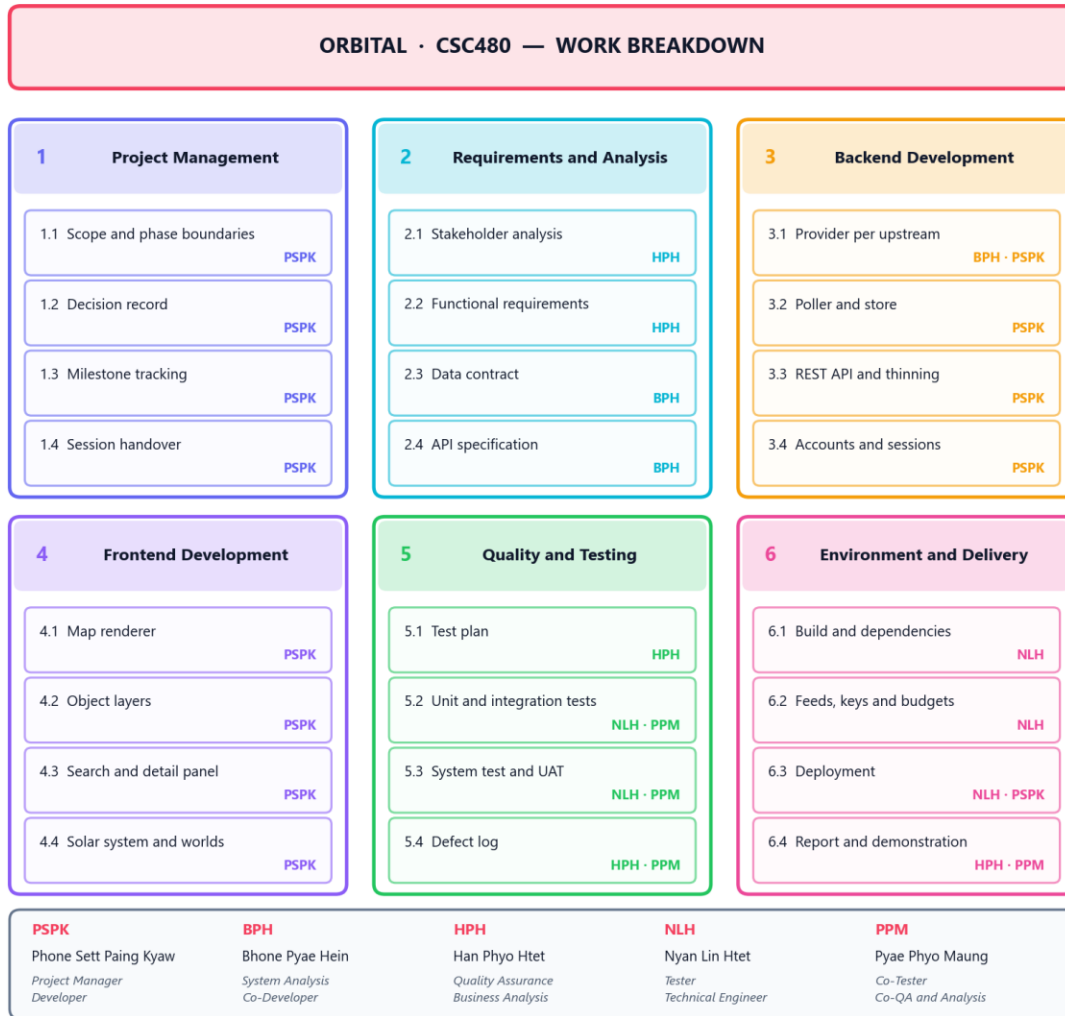


Figure 4.1 — Work breakdown structure. Six work packages, each decomposed to a deliverable with a named owner.

WBS	Work package	Primary owner	Key deliverable
1	Project Management	Phone Sett Paing Kyaw	Decision record, phase boundaries, handover
2	Requirements and Analysis	Han Phyto Htet, Bhone Pyae Hein	Requirements, data contract, API specification
3	Backend Development	Phone Sett Paing Kyaw, Bhone Pyae Hein	Providers, poller, store, REST API, accounts
4	Frontend Development	Phone Sett Paing Kyaw	Map renderer, object layers, search, detail
5	Quality and Testing	Han Phyto Htet, Nyan Lin Htet, Pyae Phyto Maung	Test plan, suites, defect log, UAT
6	Environment and	Nyan Lin Htet	Build, feeds and keys,

Delivery

deployment, report

Package 3 is decomposed by phase rather than by component, which is not the conventional choice. A component breakdown — “the ingestion layer”, “the API” — would describe the architecture rather than the work, and would hide the fact that most later packages touch the ingestion layer only by adding one file to it. Breaking it down by delivered phase makes the actual shape of the effort visible: each new object type is one provider module and one registry entry.

4.3 Project Schedule

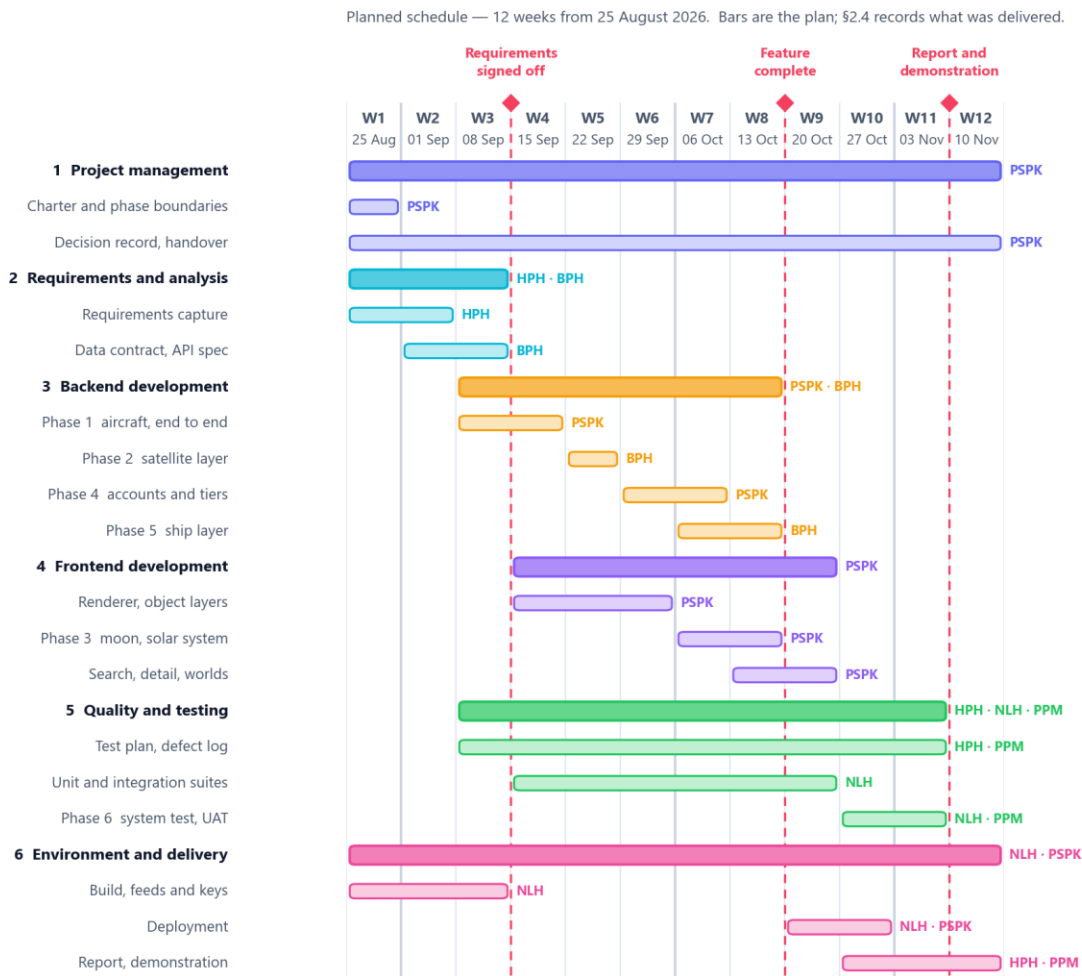


Figure 4.2 — Planned schedule. Twelve weeks, six work packages, three milestones.

This is the plan, not the log. The bars are what the team undertook to do and the order the dependencies allow. What was actually delivered, phase by phase, is recorded in §2.4, and Chapter 8 is where the two are compared. Keeping them apart matters: a Gantt chart redrawn after the fact to match what happened is not a plan and cannot be used to judge whether planning worked.

Milestones

Milestone	Week	Condition for passing
M1 Requirements signed off	3	Data contract and API specification agreed by all three layers
M2 Feature complete	8	All three object layers render live; accounts and entitlements work
M3 Report and demonstration	11	Deployed system, report, test results, demonstration rehearsed

The critical path

Requirements → data contract → the first provider → the poller and store → the API → the map renderer → the first layer on screen. Everything after that first vertical slice is parallel: the satellite layer, the ship layer and the accounts work each touch a different provider or a different route, and none of them blocks another.

That is a designed property, not a fortunate one. The shared data shape was specified before any provider was written precisely so that the second and third object types would not sit behind the first. The measured evidence that it worked is in §2.4: the ship layer cost the same as the satellite layer four months later, and the polling logic was not modified for either.

Scheduling assumption

The window is twelve weeks from **25 August 2026**, the date of the first commit. If the module's submission date differs, the plan shifts with it — the dependencies and durations do not change, only the calendar they are laid against.

4.4 Cost Estimation

Direct cost

Category	Cost
Data sources	0
Map imagery and tiles	0
Libraries and tooling	0
Hosting, development and demonstration	0
Total cash cost	0

Every input is a free public service or an open-source library, and both hosts run on free tiers. This is not an accident of a student budget: §2.2 records that free sources were a requirement, because a tracker that costs money per viewer cannot be free to look at.

The real budget is quota, not currency

No money changes hands, so the constraint that behaves like money is the upstream request allowance. It was budgeted like money:

Resource	Allowance	Planned use	Headroom
OpenSky daily credits	4,000	3,072 (77%)	928, for user-triggered lookups
adsb.lol request rate	Burst 4, then ~5/min	Within the measured limit	Enforced by a gate, not by convention
Celestrak / SatNOGS	Courtesy limits	Refreshed every six hours, cached to disk	Falls back to cache on failure
Ships (Digitraffic, aisstream)	No metered limit	Continuous websocket	—

The startup refuses a configuration projected to exceed 85% of the OpenSky allowance. A budget that is only written down is a budget nobody keeps; one the program will not start without is a budget that holds.

Labour, as a notional replacement cost

The project's only real cost is effort. Stated as what it would cost to buy, under assumptions that are stated rather than implied:

Item	Assumption
Team	5 members
Duration	12 weeks
Effort per member per week	8 hours
Total effort	480 person-hours

At a notional junior developer rate of **£25 per hour**, that is **£12,000** of labour delivered at zero cash cost. The rate is an assumption for comparison only; the hours are the figure worth arguing about, and they are an estimate, not a measurement. The project did not keep timesheets, and inventing precise ones here would be exactly the kind of invented number this report declines to produce elsewhere.

4.5 Resource Planning

People

Member	Roles	Primary work packages
Phone Sett Paing Kyaw	Project Manager, Developer	1, 3, 4
Bhone Pyae Hein	System Analysis, Co-Developer	2, 3
Han Phyo Htet	Quality Assurance, Business Analysis	2, 5, 6.4

Nyan Lin Htet	Technical Engineer, Tester	5, 6
Pyae Phyo Maung	Co-Tester, Co-QA and Analysis	5, 6.4

Seven roles across five people, so most members hold two. That is a consequence of team size rather than a design: with five people, a role per person would leave two roles unfilled, and the two that would go are quality assurance and technical environment — the two whose absence is invisible until late.

Technology

Resource	Purpose	Cost
Python, FastAPI, httpx, SGP4	Ingestion and API	Free, open source
React, TypeScript, MapLibre GL, Vite	Browser client	Free, open source
SQLite	Accounts and sessions	Free, bundled
Vercel	Frontend and landing page hosting	Free tier
Northflank	Backend container hosting	Free tier
GitHub	Source control, history, issues	Free

Data

Five upstream feeds, all free and public: adsb.lol and adsb.fi for aircraft, Celestrak and SatNOGS for orbital elements and satellite identity, Digitraffic and aisstream for vessels. OpenSky remains supported in the code and is not in use.

Environment

A member must be able to set the project up from a clean machine. That is the Technical Engineer's deliverable rather than a shared assumption, and it is the reason generated assets are fetched by a script rather than committed: a clone plus one command produces a running system, and a build with no network works as long as a previous one has run.

4.6 Risk Management Plan

Probability and impact are graded **H/M/L**. The response column says what was actually built or done, not what would ideally be done — a risk register of intentions is a document nobody checks against the system.

#	Risk	P	I	Response
R1	An upstream feed becomes unavailable	H	H	Last good snapshot served with a visible staleness flag; aircraft and ships each have two independent feeds

R2	An upstream rate-limits or bans the client	M	H	One backend for all viewers; measured limits enforced by a gate; exponential backoff honouring Retry-After
R3	The browser cannot draw the object count	M	H	Responses thinned to a 2,000 cap; one layer at a time; positions interpolated between polls
R4	Traffic outgrows the single backend	M	H	Viewer count is decoupled from upstream cost; conditional requests answer roughly twenty-nine polls in thirty without building a response; per-request work is bounded by the 2,000 cap
R5	The map shows something false and nothing says so	M	H	Every response carries its age and a staleness flag; a marker fades after two minutes unreported; the status bar states how many of how many are drawn
R6	Secrets committed to the repository	L	H	Credentials in environment variables only; configuration never printed as an object, only field by field
R7	Work serialises behind one member	M	M	The shared data shape fixed before any provider was written, so the layers proceed in parallel; handover written every session
R8	A feed's licence forbids the intended use	M	M	Licences recorded per source; the commercial constraint is stated in §2.2 rather than assumed away
R9	A defect resists diagnosis and consumes the schedule	M	H	Every defect written down with how it was found; a fix counts as fixed when demonstrated, not when written
R10	A decision correct when made becomes wrong later	M	M	Decision record carries alternatives and reasoning, so a reversal is cheap and evidenced

Ordered by how much each one shaped the system, rather than by probability times impact. R1 to R5 are the ones with a *structural* mitigation — something in the architecture exists because of them, and would not exist otherwise:

- **R1** is why a provider can be two providers. The union hides the pair behind one interface, so a feed disappearing costs one module.
- **R2** is why there is exactly one backend. A hundred open tabs cost the same upstream quota as one, because the browser never calls a source.
- **R3** is why responses are capped at 2,000, why one object layer is drawn at a time, and why positions are interpolated between polls rather than fetched.
- **R4** is why the list endpoint answers conditional requests, and why the ETag is computed from the store's version counter *before* the store is read.

- **R5** is why the status bar exists in the form it does, why a marker fades rather than simply staying put, and why a provider raises instead of returning what it managed to get.

The remaining five are managed by practice rather than by structure — a credential kept out of a file, a defect written down, a decision recorded. That is a weaker kind of mitigation, because it depends on somebody continuing to do it, and that is why they come second.

R4 in detail, because it is the one with a ceiling

The client polls every 10 seconds and tier 1 refreshes every 300, so roughly **twenty-nine polls in thirty ask for a snapshot that has not changed**. Each of those would otherwise serialise two thousand objects, gzip them and write them to a socket — on the single event loop the poller runs on. The ETag turns them into a header comparison, and because the tag is computed from the store's version counter and the query before the store is read, a 304 never builds the response at all.

The one poll in thirty that does change costs the measured work in §2.1: a bounding-box filter and a thinning pass, together under one and a half milliseconds at the 2,000 cap.

Upstream cost does not move at all with viewer count, because the browser never calls a source.

Where the ceiling is, honestly. Scaling is vertical only. A second worker would open a second AIS websocket, run its own poller and keep its own store — doubling the upstream cost and letting two requests disagree about where an aircraft is. So the answer to sustained growth is a bigger box until a bigger box runs out, and past that the design has to change: a shared store the workers read, which is a different system from the one described in Chapter 5. That is a known limit rather than an oversight, and it is stated here because a risk register that records only the risks with comfortable answers is not one.

R5 is the one a tracker has to take seriously

Every other risk here is visible when it happens: a blank layer, a slow page, a failed deploy.

This one is invisible by construction. A map showing positions from four minutes ago looks exactly like a map showing positions from four seconds ago, and a sample of two thousand aircraft looks exactly like all of them. The user cannot detect the failure, so the system has to declare it.

That is why Objective O5 — honest reporting of what is shown — is an objective rather than a nicety, and why the status bar reports sampling, age and source rather than leaving them to be assumed.

R1 was realised, and the plan is judged on that

OpenSky became unreachable from every cloud host during the project. The mitigation was not theoretical: a second aircraft feed was added, the union provider kept the interface identical, and **nothing above the ingestion layer changed**. The cost was one provider module and one registry entry — the same cost the architecture had been designed to make it.

R9 was also realised, and cost more

Six working sessions went to a single defect that was misdiagnosed five times. The cause was structural — one camera serving two pictures at very different scales — and the resolution deleted 376 lines, added 130, and cost no feature. It is recorded here because a risk register that lists only the risks that were survived cheaply is not a risk register.

4.7 Communication Plan

Channel	Participants	Frequency	Purpose
Written handover	Whole team	End of every working session	What was done, what is blocked, who is waiting on whom
Decision record	Author, reviewed by PM	On every significant choice	The choice, its alternatives, and the reasoning
Defect log	QA, Testers, Developer	On every defect found	Severity, how it was found, whether it is fixed
Repository history	Whole team	Continuous	The change, and the reason for it, in the commit message
Team review	Whole team	Per phase boundary	Accept the phase, or state what is outstanding
Report and demonstration	Whole team, assessor	Milestone M3	The deliverable

Why handover is a deliverable rather than a courtesy

The single largest risk to a part-time team is not technical: it is that context is lost between sessions and re-derived at full cost. Every session therefore ends with a written handover, and it is package 1.4 in the WBS with an owner against it — because a practice that is only a good intention is a practice that stops the first week somebody is busy.

Escalation

A blocked item is raised in the handover. If it remains blocked at the next session it goes to the Project Manager, who decides whether it is in the current phase at all. **A blocker that is out of the phase is closed rather than carried**, which is how the phase boundary does its job: the alternative is a list that grows all semester and is resolved by the deadline rather than by anyone.

Chapter 5: System Design

5.1 System Architecture

Orbital is three layers with one rule between them: **data flows in one direction, and each layer knows only the layer directly beneath it.**

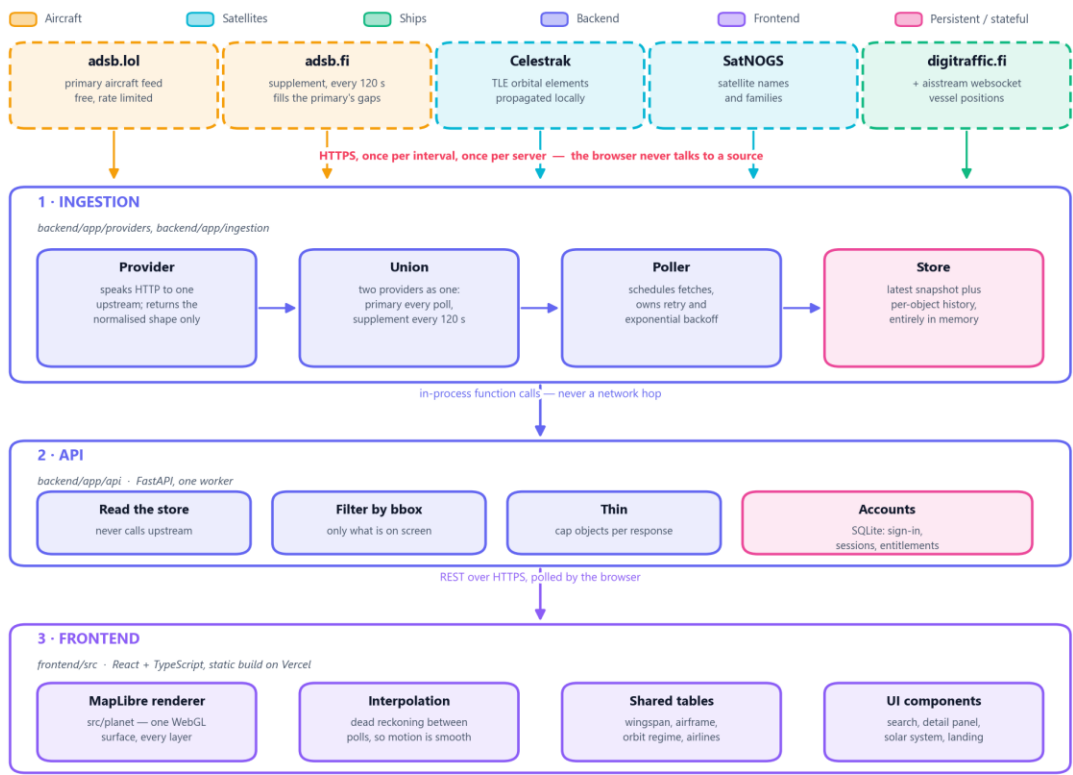


Figure 5.1 — System architecture. Five external feeds, three internal layers, and one direction of flow.

The three layers

Layer	Owns	Never does
1 Ingestion	Speaking to upstreams, normalising, scheduling, holding the current state	Know that a browser exists
2 API	Reading the store, filtering by bounding box, thinning, accounts	Call an upstream
3 Frontend	Rendering, interpolation, interaction	Know where the data came from

Three properties follow, and they are the design

The browser never talks to a data source. Every external call goes through the backend, so rate limiting and caching are enforced in exactly one place. A hundred open browser tabs cost the same upstream quota as one. This is what made adding a second aircraft feed a backend-only change: the frontend was never told, because there was nothing to tell it.

Either end can be replaced without touching the other, and both have been. The backend gained a second upstream and the frontend gained an entirely different renderer, neither requiring a change on the other side of the wire.

Upstream failure is contained at layer 1. The API serves the last good snapshot with an explicit staleness flag. A feed outage degrades the display; it does not break it.

Deployment

Component	Host	Shape
Frontend	Vercel	Static build, no server
Landing page	Vercel	/landing, the same deployment, static files
Backend	Northflank	One container, one worker
Accounts database	Northflank volume	SQLite file

One worker, deliberately. The backend holds a websocket open to the AIS stream, polls on a schedule, and answers every request from memory. A second worker would open a second AIS socket, run its own poller and keep its own store: the upstream cost would double and two requests could disagree about where an aircraft is. It is scaled by making the box bigger, not by adding boxes.

The numbers that shape it

Parameter	Value	Why that value
Objects per response	2,000	The cap above which the browser cannot draw a frame in budget
Object retention	300 s	Longer than the longest poll interval, so a missed poll never drops an aircraft
Union supplement interval	120 s	The metered feed answers once per interval regardless of the free feed's cadence
Aircraft request rate	4 per minute	A fifth under the measured limit of burst-4 then ~5 per minute
Vessel ceiling	14,000	Bounds memory for a feed that accumulates rather than answers

5.2 Database Design

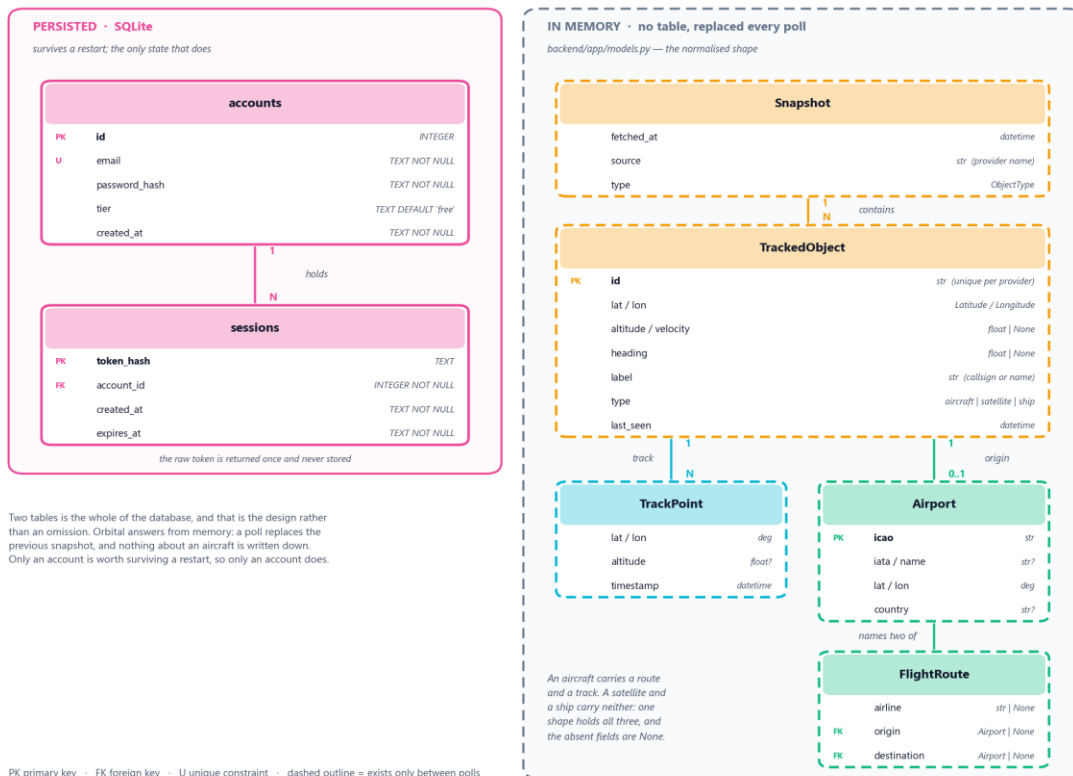


Figure 5.2 — Entity relationship diagram. Two persisted tables, and the domain entities that have no table.

Two tables, and that is the design

The persisted schema is `accounts` and `sessions`. Nothing else in Orbital is written down.

```
CREATE TABLE accounts (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  email       TEXT     NOT NULL UNIQUE,
  password_hash TEXT   NOT NULL,
  tier         TEXT     NOT NULL DEFAULT 'free',
  created_at  TEXT     NOT NULL
);

CREATE TABLE sessions (
  token_hash TEXT     PRIMARY KEY,
  account_id INTEGER NOT NULL,
  created_at TEXT     NOT NULL,
  expires_at  TEXT     NOT NULL
);

CREATE INDEX sessions_account ON sessions (account_id);
```

An account holds many sessions: one per signed-in device, each expiring on its own schedule.

The raw session token is returned once, at creation, and never stored. Only its hash is in the table, so the token exists in the user's cookie and nowhere else. A copy of the database does not let anyone sign in as anybody.

Why almost nothing is persisted

A tracker looks like a system that should have a large database, and Orbital deliberately does not. A poll replaces the previous snapshot; positions are held in memory and served from there; an object not re-observed within the retention window is evicted. Nothing about an aircraft is written to disk at all.

Three consequences, and all three are wanted:

- **The API cannot be slow because a database is slow**, because there is no database on the request path for tracked objects.
- **There is no schema migration** for the thing that changes most often — the shape of a feed — because a feed's shape is normalised at the boundary and never stored in its own form.
- **Restarting is cheap and safe.** The only state worth keeping is an account, and that is the only state kept.

The cost is equally plain: Orbital cannot answer a question about last Tuesday. Entitlement windows of twenty-four hours and seven days are bounded by what is still in memory, not by a query.

The entities that have no table

`TrackedObject` is the shared shape all three object types normalise to, with `TrackPoint`, `Airport` and `FlightRoute` hanging off it. They have real structure, real relationships and real cardinality — they simply have no rows. They are drawn dashed in Figure 5.2 because omitting them would make the data model look like two tables about sign-in, which is the least interesting part of this system.

One shape holds all three object types, and the absent fields are `None`. An aircraft has a route and a track; a satellite and a ship have neither. This is what makes a new object type cost one ingestion module: there is no per-type schema to add.

5.3 UML Diagrams

5.3.1 Class Diagram

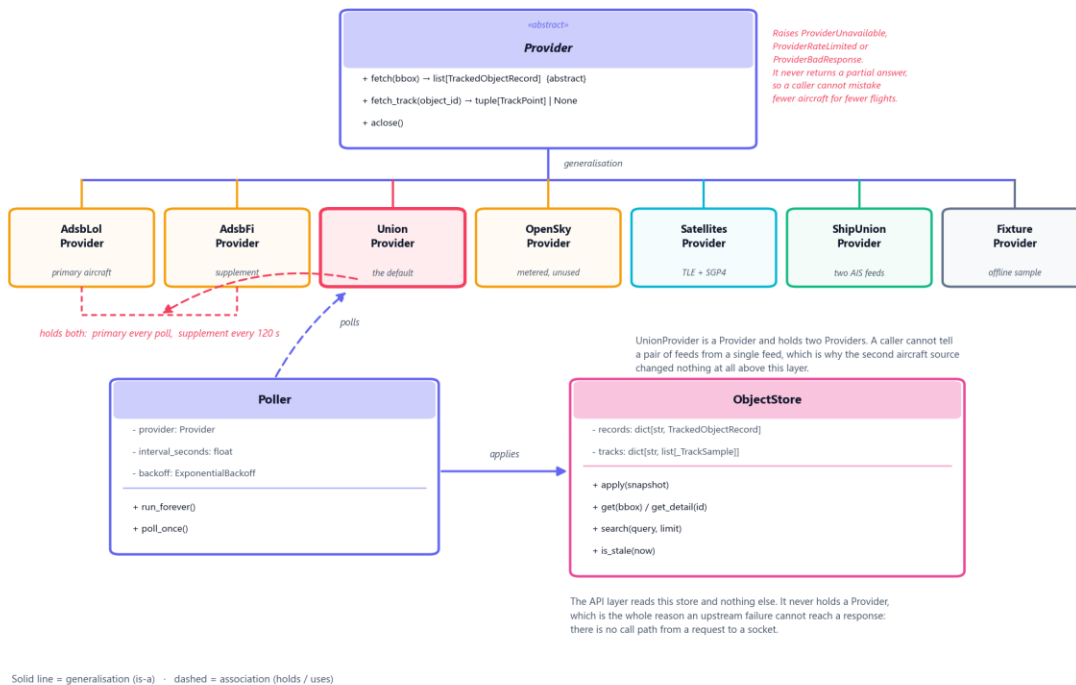


Figure 5.3 — Class diagram of the ingestion layer.

The ingestion layer is the part of Orbital with real class structure, so it is the part worth drawing. `Provider` is abstract and declares one required operation: `fetch`, given an optional bounding box, and return the normalised shape. Seven concrete providers implement it — one per upstream, plus an offline fixture that needs no credentials.

UnionProvider is the design point. It inherits `Provider` and holds two of them: the primary answers every poll, the supplement answers every 120 seconds, and the results are merged. A caller cannot tell a pair of feeds from a single feed.

That is why the second aircraft source cost what it cost. When the primary became unreachable from every cloud host, the change was one provider module and one registry entry; the poller, the store, the API and the entire frontend were untouched, because none of them can observe the difference.

`Provider` also raises rather than returning partial results — `ProviderUnavailable`, `ProviderRateLimited`, `ProviderBadResponse`. A provider that returned nine hundred aircraft instead of two thousand would be reporting that the sky had emptied. Raising makes a failure a failure.

The API never holds a `Provider`. There is no call path from an HTTP request to a socket, which is the structural reason §5.1's third property is true rather than merely intended.

5.3.2 Sequence Diagram

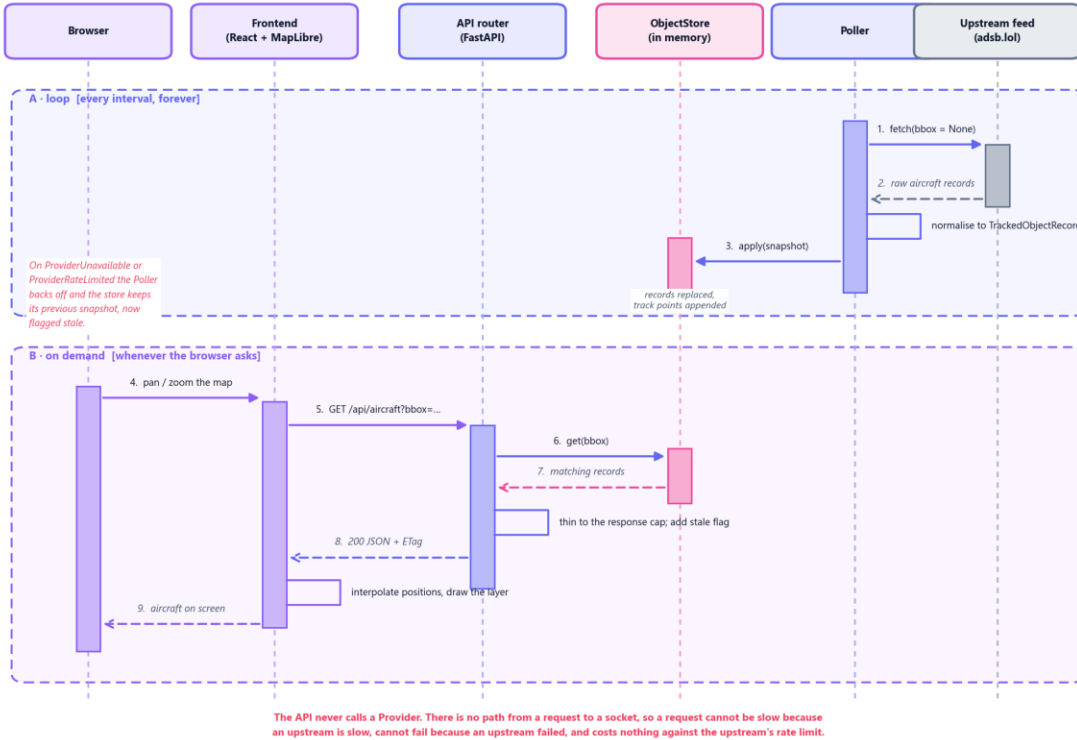


Figure 5.4 — Sequence diagram. The poll loop, and a request that does not wait for it.

Two interactions are drawn on one page deliberately. Separately they look like an ordinary background job and an ordinary request; together, the actual design is visible — **they never touch**.

A. The poller wakes on its interval, asks the provider to fetch, normalises what comes back, and applies it to the store. This runs whether or not anyone is looking.

B. The browser pans the map and asks for a bounding box. The API reads the store, filters, thins to the response cap, adds the staleness flag, and answers. The frontend interpolates positions between polls so that motion is smooth at a far lower request rate than smooth motion would otherwise need.

The browser's request does not trigger a fetch, does not wait for one, and cannot fail because one failed. It reads whatever the store last had. That single property is why an upstream outage degrades Orbital rather than breaking it, and why viewer count is decoupled from upstream cost entirely.

5.3.3 Activity Diagram

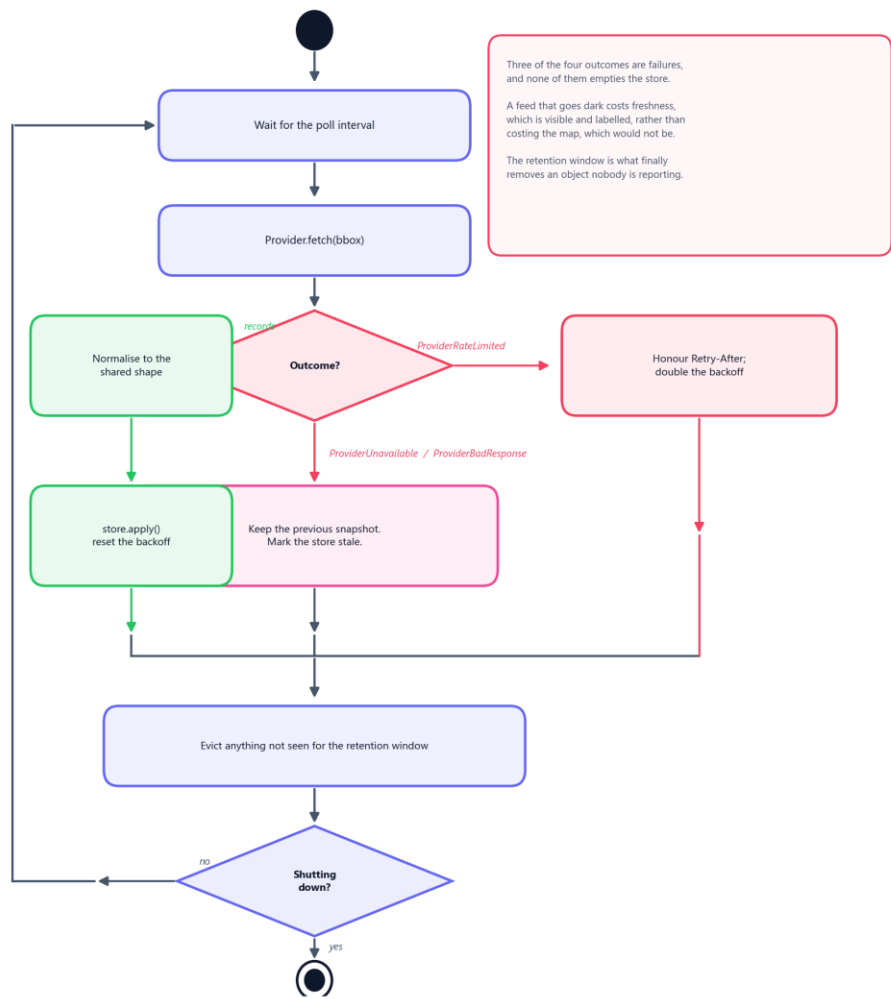


Figure 5.5 — Activity diagram. One poll cycle, including the ways it fails.

An activity diagram of the happy path would be a straight line and would say nothing. The branches are the content: **three of the four outcomes of a fetch are failures**, and what happens to each is the reason the map does not go blank when a feed does.

Outcome	Response
Records returned	Normalise, apply to the store, reset the backoff
ProviderRateLimited	Honour <code>Retry-After</code> , double the backoff, keep the snapshot
ProviderUnavailable	Keep the previous snapshot, mark the store stale
ProviderBadResponse	As above — a malformed answer is treated as no answer

None of the three failures empties the store. A feed that goes dark costs freshness, which is visible and labelled in the status bar, rather than costing the map, which would not be. The retention window is what finally removes an object that nobody is reporting — a decision made by elapsed time, not by one failed request.

5.4 User Interface Design

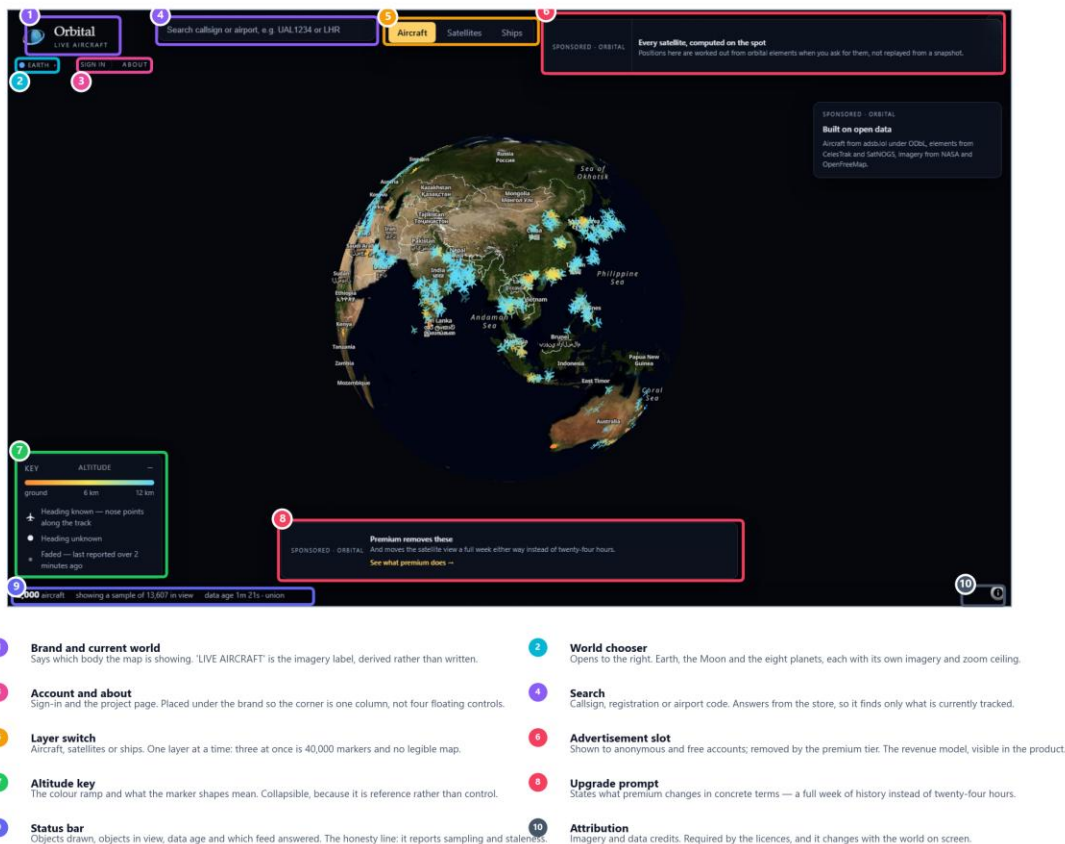


Figure 5.6 — The deployed interface, annotated. Captured headlessly at 1600 × 950 while the deployment was live.

This is a screenshot of the running system rather than a wireframe. The counts in the status bar — objects drawn, objects in view, data age, and which feed answered — are the real ones at the moment of capture.

The principles the layout follows

The map is the product; everything else is chrome at the edges. Controls occupy the corners and the top strip. Nothing floats over the centre, because the centre is the thing the user came for.

One layer at a time. Aircraft, satellites and ships are a switch rather than three checkboxes. All three at once is roughly forty thousand markers and no legible map; the switch makes that impossible rather than merely discouraged.

The corner is one column, not four floating controls. Brand, world chooser, sign-in and about stack under one another, and the world chooser opens to the right rather than downward so it does not cover the items beneath it.

The status bar is the honesty line. It states that 2,000 of 13,607 objects in view are drawn, how old the data is, and which feed answered. A tracker that quietly showed a sample as though it were everything would be easier to build and would be lying.

Colour carries meaning, and only meaning. Altitude is a ramp; marker shape distinguishes a known heading from an unknown one; a faded marker has not been reported for over two minutes. The key states all three rather than expecting the reader to infer them.

The revenue model is visible in the product. Advertisement slots are shown to anonymous and free accounts and removed by the premium tier, and the upgrade prompt says what premium changes in concrete terms — a full week of history instead of twenty-four hours — rather than in adjectives.

5.5 Prototype Design

What the prototype is

The prototype is the deployed system. There is no separate mock-up, and that is a deliberate consequence of how the work was sequenced: the first phase delivered one object type end to end — feed, normalisation, store, API, map, marker on screen — rather than delivering a layer at a time across all three.

A vertical slice is a prototype that is also the first increment of the product. The alternative, a horizontal one, produces a complete ingestion layer with nothing to look at, and defers every integration risk to the end.

	Address
Application	<code>orbital-liveview.vercel.app</code>
Landing page	<code>orbital-liveview.vercel.app/landing</code>

What each phase’s prototype proved

Phase	Prototype	The risk it retired
1	Aircraft, end to end	That a free feed could sustain a live map at all
2	Satellites; migration to MapLibre	That a second object type costs one module
3	Moon and solar system	That the renderer is not Earth-specific
4	Accounts, tiers, advertisements	That a revenue model fits without a payment integration
5	Ships, regional then global	That the shape holds for a streaming source, not just a polled one
6	Performance and defect sweep	That the whole thing holds at full object count

Evolution rather than throw-away

Each phase's prototype became the next phase's foundation, and none was discarded. The evidence that this was the right shape is the cost of the later phases: the satellite layer cost one provider module and one registry entry, and the ship layer cost the same four months later, with the polling logic unmodified for either.

One prototype was discarded, and it is worth recording. The original renderer was a three.js globe, replaced by MapLibre in phase 2. The globe could draw a sphere but could not draw a map: no vector basemap, no zoom to street level, no established tiling. Replacing it cost a phase and was correct — a decision that was right when it was made became wrong once the requirement grew past what it could reach.